

MTree e t8code no HiGFlow

Como construir, rodar e verificar o solver com os dois backends de malha. Todos os comandos desta página foram executados como estão escritos.

O HiGFlow sempre leu a malha de um arquivo AMR e a distribuiu com o `lbal`. Agora existe um segundo caminho: a malha pode ser produzida pelo **t8code**, e a partição também. Nada disso é obrigatório — **sem pedir, nada muda**, e o t8code não é sequer dependência.

Este tutorial cobre o que funciona hoje e diz, em cada ponto, o que *não* funciona. Onde há limite, ele está declarado — não descoberto depois.

1. Construir

O ambiente sai do `varsrc`, que precisa ser carregado **da raiz do repositório**:

```
# sempre a partir da raiz: o varsrc usa $(pwd)
cd /caminho/para/HiGFlow
set -a; . ./varsrc; set +a
export HIGTREE_DIR="$PWD/higtrees" HIGFLOW_DIR="$PWD/higflow"
```

O **make clean** **não é opcional ao trocar de dimensão**. Os objetos não carregam a dimensão no nome, então depois de um build em `DIM=3` os arquivos parecem atuais para um build em `DIM=2`: o make não reconstrói nada e o binário 2D liga objetos 3D. Isso falha como segfault e números errados, não como erro de compilação.

```
make -C higtrees clean && make -C higtrees DIM=2
make -C higflow clean && make -C higflow DIM=2
make -C higflow/example2d_Newt
```

Com o t8code (opcional)

O t8code não está empacotado em distribuição nenhuma; vem do fonte. O `libp4est-dev` do Debian existe, mas não basta — o CMake do t8code busca `sc` e `p4est` por `FetchContent`.

```
cd bibliotecas
git clone https://github.com/DLR-AMR/t8code.git
cd t8code && git checkout v4.0.0-26.09
cmake -B build -S . \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$PWD/install" \
  -DT8CODE_ENABLE_MPI=ON \
  -DT8CODE_BUILD_AS_SHARED_LIBRARY=ON \
  -DT8CODE_BUILD_TESTS=OFF -DT8CODE_BUILD_EXAMPLES=OFF \
  -DT8CODE_BUILD_BENCHMARKS=OFF -DT8CODE_BUILD_TUTORIALS=OFF
cmake --build build -j4 && cmake --install build
```

Não clone com `--depth 1`. O CMake deriva a versão de `git describe`, e sem as tags ele para com `VERSION ".." format invalid` — mensagem que não menciona tag nenhuma. Se já clonou raso: `git fetch --unshallow --tags`.

Depois, o exemplo ganha as fontes alternativas passando o prefixo:

```
make -C higflow/example2d_Newt clean
make -C higflow/example2d_Newt T8CODE="$PWD/bibliotecas/t8code/install"
```

2. Rodar

Sem variável de ambiente, o exemplo lê o arquivo AMR — o caminho de sempre:

```
cd higflow/example2d_Newt
mpirun -n 2 --oversubscribe ./ns-example \
  SAIDA/example-2d.load SAIDA/out.save SAIDA/VTKS/p.print \
  -ksp_type bcgs -pc_type bjacobi
```

Com `HIGFLOW_MALHA`, a malha passa a vir do t8code:

```
HIGFLOW_MALHA=t8code-particao mpirun -n 2 --oversubscribe ./ns-example ...
```

O diretório `VTKS` precisa existir antes; o solver não o cria. E copie os arquivos de `input/` para o diretório de saída — é deles que o `.load` é lido.

3. As quatro fontes

Cada uma exercita um estágio diferente. As três primeiras entregam árvores para o `lbal` reparticionar; a última entrega o resultado já particionado.

HIGFLOW_MALHA	QUEM DECIDE A MALHA	QUEM DECIDE A PARTIÇÃO	LIMITE
(não definida)	arquivo AMR	lbal	—
t8code	t8code, em série	lbal	malha uniforme; N blocos
t8code-rank	t8code, por rank	lbal	malha uniforme; 1 bloco
t8code-particao	t8code	t8code	malha uniforme; 1 bloco

Por **rank** significa que nenhum processo chega a ter a malha inteira na memória — era esse o gargalo que motivou o t8code. A região de um rank vira um *conjunto de caixas completas*, porque árvore com buracos não é navegável: `hig_get_cell_with_point` desreferencia filho nulo.

4. Verificar

Há duas suítes, e elas cobrem coisas diferentes.

SUÍTE DE EXEMPLOS — FÍSICA CONTRA REFERÊNCIA GRAVADA

```
python3 ci/run_suite.py # 33 de 33, só o caminho AMR
python3 ci/run_suite.py --t8code bibliotecas/t8code/install
# 73 de 73, AMR + variantes t8code
```

Com `--t8code`, cada caso elegível roda também pelas fontes que declara, **contra a mesma referência**. É isso que dá valor ao modo: malha uniforme produzida por outro backend tem de dar o mesmo número. Nenhuma referência nova foi gerada a partir do código novo — isso só guardaria contra regressão futura.

SUÍTE DA BIBLIOTECA — CONTRATO DE MALHA

```
python3 ci/run_hightree_tests.py # 170/170, contrato 18/18
python3 ci/run_hightree_tests.py --t8code bibliotecas/t8code/install
# 316/316, contrato 18/18
```

O contrato vive em `hightree/src/hig-mesh-contract.h` e não é prosa: o driver lê as cláusulas do próprio cabeçalho e exige que cada uma tenha caso passando. Cláusula sem teste sai como `SEM TESTE QUE RODE`; cláusula cujo teste falhou, como `REPROVADA`. As duas contam como falha.

ORÁCULO DO INSTANTÂNEO — LIGADO POR AMBIENTE

```
HIGTREE_VERIFY_SNAPSHOT=1 python3 ci/run_suite.py
```

Com essa variável, **todo domínio montado** passa pelo oráculo diferencial — são 19 por execução do solver — e divergência aborta. Ele localiza cada célula *por ponto*, e não percorrendo: o instantâneo foi preenchido percorrendo o iterador e indexando pelo mapeador, e o mapeador saiu desse mesmo iterador. Conferir um contra o outro mediria a aritmética contra ela própria.

5. Quais exemplos servem

A fonte t8code reproduz malha **uniforme**. O levantamento dos arquivos de domínio:

SITUAÇÃO	QUANTOS	QUAIS
Um bloco uniforme	8	Newt, Oldroyd, Gptt, VOF, VOF_Gptt, VOF_Oldroyd, BMP, KBKZ
Dois blocos uniformes	1	Newt_contraction
Sem arquivo de domínio	5	ElectroOsmotic, DynamicMeshAdapt, os dois 3D, examples-common

Sete estão ligados. O `BMP` ficou de fora porque inicializa o domínio por outro caminho, e o `KBKZ` está marcado como quebrado na suíte. Ligar um exemplo custa uma linha (`malha_t8_instala(ns, myrank)`) mais o bloco `T8CODE` no Makefile.

Com malha refinada a fonte **recusa** em vez de aproximar. Malha diferente dá números diferentes, e o silêncio ali seria pior que o erro.

6. Armadilhas

Todas custaram tempo de verdade nesta árvore. Estão aqui porque nenhuma se anuncia com clareza.

Build que falha sem dizer error:

A guarda do Makefile para com `*** Biblioteca ausente: ... libhig2d.a` quando a árvore está na outra dimensão. Isso **não** casa com `grep error:`, então um build falho passa por bem-sucedido e o teste roda o binário velho. Confira por status de saída:

```
if make -C higtrees/tests DIM=2 T8CODE=... alvo-2d >/dev/null 2>&1; then ...
```

O sintoma para reconhecer: *um resultado que não muda quando deveria mudar.*

O varsrc carregado do diretório errado

Ele usa `$(pwd)`. Carregado de dentro de `higtrees/tests`, aponta o `PETSC_DIR` para um `bibliotecas/` que não existe ali. O sintoma é `undefined reference to PetscFinalize` ao ligar — que aponta para o PETSc, não para o diretório errado.

Comparação byte a byte só vale em np=1

O preconditionador `bjacobi` depende da partição, então a partição *sozinha* move os números: o caminho AMR comparado consigo mesmo em np=1 contra np=2 já difere em 3 dos 6 campos gravados. Em np>1 o oráculo é a referência gravada, com tolerância — para qualquer fonte de malha.

Antes de acusar o código novo, rode o baseline

Duas vezes uma reprovação contra a referência parecia defeito do t8code e era configuração de execução: a referência é de `numsteps=20` e `dtp=0.005`, que a suíte sobrescreve e uma execução manual não. Nas duas, o caminho AMR reprovava igual.

7. O que ainda não existe

- **Malha com refino pelo t8code, ponta a ponta.** A faixa do remetente e a árvore do receptor precisariam concordar na subdivisão interna — é trocar estrutura, não índices.
- **Os exemplos sem arquivo de domínio** constroem a malha por outro caminho e não têm fonte alternativa.
- **Código sem cobertura.** 207 laços foram migrados para o instantâneo em arquivos que nenhum teste executa. Eles compilam e a transformação é a mesma que a suíte validou onde havia cobertura — mas isso é o que se afirma, e

nada além.

Branch `mesh-t8code` . Números desta página: HiGTree 170/170 sem t8code e 316/316 com; HiGFlow 33/33 sem a flag e 73/73 com; contrato de malha 18/18 nas duas configurações.